VECTOR

VHSM CORE

Technical Reference

vHsm

Version 5.0.1

Authors	vistof, visdil
Status	Released

Document Information

History

Author	Date	Version	Remarks
vistof	2019-03-03	2.1.0	Initial creation of Technical Reference
vistof	2019-08-15	3.0.0	Add Callouts in chapter 5.4.1 Describe new key storage behavior in chapter 3.4.5 Startup Sequence updated (2.4)
vistof	2019-09-10	3.0.1	Add descriptions of callouts (5.4.1.17, 5.4.1.4)
vistof	2019-10-31	3.1.0	Add event log(3.4.10.4) Add cyclic secure boot verification (3.4.10.8.6)
vistof	2020-01-31	4.0.0	Add Multiple Task/Os description (3.4.8) Add Communication Register description (3.4.2)
visdil, vistof	2020-06-19	5.0.0	Add allow/disallow key usage api (5.2.9, 5.2.10) Add prepare/exit ram loop api (5.2.17, 5.2.18) Add support for HASH in addition to CMAC for secure boot (see 3.4.10.8) Add chapter for storage method (see 3.4.10.8.1) Remove Write-Once Key Add vHsm_PreRamLoopCallout (5.4.1.12), Adapt description of vHsm_RamLoopCallout (5.4.1.11) Add Ramloop section (3.4.7)
vistof	2020-12-10	5.0.1	Fix naming of CRYPTO_ALGOFAM_CUSTOM_VHSM_SECURE_BOOT

Reference Documents

No.	Source	Title	Version
[1]	VECTOR	TechnicalReference_vHsm_Core.pdf	See delivery
[2]	VECTOR	TechnicalReference_vHsm_Custom.pdf	See delivery
[3]	VECTOR	TechnicalReference_vHsm_Hal.pdf	See delivery
[4]	VECTOR	TechnicalReference_Crypto_30_LibCv.pdf	See delivery
[5]	VECTOR	vHsm_IntegrationHints.pdf	See delivery
[6]	AUTOSAR	AUTOSAR_CryptoDriver.pdf	4.3.0
[7]	AUTOSAR	AUTOSAR_CryptoDriver.pdf	4.4.0

© 2020 Vector Informatik GmbH



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

© 2020 Vector Informatik GmbH

Contents

1 Component History 10

2 Introduction 11
2.1 Architecture Overview 11
2.2 vHsm Architecture Overview 12
2.3 vHsm Job Processing Overview 14
2.4 vHsm Startup Sequence Overview 15

3 Functional Description 16
3.1 Features 16
3.1.1 Deviations 16
3.1.2 Additions/ Extensions 16
3.2 Initialization 17
3.3 Error Handling 17
3.3.1 Development Error Reporting 17
3.4 vHsm 17
3.4.1 Pre Config File 17
3.4.2 Communication Registers 17
3.4.2.1 HSM2HOST 17
3.4.2.2 HOST2HSM 17
3.4.3 IPC Versions 18
3.4.4 Memory Access 19
3.4.5 Key Storage 19
3.4.6 Interrupts 19
3.4.7 Ramloop 20
3.4.8 Multiple tasks 20
3.4.8.1 Processing inside ISR 21
3.4.8.2 Os 21
3.4.9 Custom definitions 22
3.4.9.1 Custom Key Elements 22
3.4.9.2 Custom Key Element Values 22
3.4.9.3 Custom Algorithm Family 23
3.4.9.4 Custom Algorithm Mode 23
3.4.10 Custom Services 23
3.4.10.1 Version Info 24
3.4.10.2 UID 24
3.4.10.3 Error Log 24
3.4.10.4 Event Log 24

3.4.10.5 Lazy Persist Keys 24
3.4.10.6 Write Once Keys 24
3.4.10.7 Repersist Keys 24
3.4.10.8 Secure Boot 25
3.4.10.8.1 Storage Method 25
3.4.10.8.2 Verification Method 25
3.4.10.8.3 Secure Boot Segment 25
3.4.10.8.4 Secure Boot Group 26
3.4.10.8.5 Automatic start of secure boot 26
3.4.10.8.6 Cyclic Secure Boot Verification 26
3.4.10.8.7 Manual trigger of secure boot groups and segments 26
3.4.10.8.8 Indicate the end of Secure Boot 27
3.4.10.8.9 Secure boot results 27
3.4.10.9 Secure Software Download 28
3.4.10.10 vHsm Secure Boot Update 28
3.4.10.11 vHsm Firmware Update 29
3.4.11 Limitations 30
3.4.11.1 Secure Boot 30
3.4.11.2 Platforms with memory windows 30

4 Integration 31
4.1 Embedded Implementation 31
4.2 Integration Hints 32

5 API Description 33
5.1 Services provided by CRYPTO driver 33
5.1.1 vHsm_Core_ProcessJob 33
5.1.2 vHsm_Core_CancelJob 33
5.1.3 vHsm_Core_KeyElementSet 34
5.1.4 vHsm_Core_KeyElementGet 35
5.2 Services provided by vHsm_Core 35
5.2.1 vHsm_Api_KeyValidSetAndPersist 35
5.2.2 vHsm_Api_Local_KeyElementGet 36
5.2.3 vHsm_Api_InitializeTempJob 37
5.2.4 vHsm_Api_ProcessTempJob 37
5.2.5 vHsm_Api_CancelTempJob 38
5.2.6 vHsm_Api_KeyElementGetForCryptoJobs 39
5.2.7 vHsm_Api_KeyElementSet 39
5.2.8 vHsm_Api_KeyElementSet_Unchecked 40
5.2.9 vHsm_Api_DisallowKeyUsage 41

5.2.10 vHsm_Api_AllowKeyUsage 41
5.2.11 vHsm_Init 42
5.2.12 vHsm_TriggerSoftwareReset 42
5.2.13 vHsm_TriggerSoftwareResetWithIpcClearing 43
5.2.14 vHsm_WriteErrorLog 43
5.2.15 vHsm_ExternalIsr 44
5.2.16 vHsm_InternalIsr 44
5.2.17 vHsm_Api_Ramloop_PrepareExecution 45
5.2.18 vHsm_Api_Ramloop_Exit 45
5.3 Services used by vHsm_Core 46
5.4 Configurable Interfaces 47
5.4.1 Callout Functions 47
5.4.1.1 vHsm_StartupCallout 47
5.4.1.2 vHsm_SystemInit_Callout 47
5.4.1.3 vHsm_FirstStartup_Callout 48
5.4.1.4 vHsm_InvalidBlockDetected_Callout 48
5.4.1.5 vHsm_InitOne_Callout 49
5.4.1.6 vHsm_InitTwo_Callout 49
5.4.1.7 vHsm_InitThree_Callout 50
5.4.1.8 vHsm_NvM_ProcessBlock_Callout 50
5.4.1.9 vHsm_NvMainFunctions_Callout 51
5.4.1.10 vHsm_MainFunctionCyclic_Callout 51
5.4.1.11 vHsm_RamLoop_Callout 52
5.4.1.12 vHsm_PreRamLoop_Callout 52
5.4.1.13 vHsm_FirmwareUpdate_GetMacKey_Callout 53
5.4.1.14 vHsm_FirmwareUpdate_GetCipherKey_Callout 53
5.4.1.15 vHsm_Custom_IsSecureBootAllowAutostart_Callout 54
5.4.1.16 vHsm_Custom_IsSecureBootAllowCyclicVerification_Callout 54
5.4.1.17 vHsm_Custom_SecureBootSanction_Callout 55
5.4.1.18 vHsm_SecureBoot_SegmentSuccess_Callout 56
5.4.1.19 vHsm_Log_Event_Callout 56
5.4.1.20 vHsm_Log_FillLevelThreshold_Callout 57
5.4.1.21 vHsm_Log_GetTimestamp_Callout 57

6 Configuration 58
6.1 Configuration Variants 58
6.2 Configurable Attributes 58

7 Glossary and Abbreviations 59
7.1 Glossary 59

7.2 Abbreviations 59

8 Contact 60

Illustrations

Figure 2-1 AUTOSAR 4.3 Architecture Overview 11
Figure 2-2 AUTOSAR 4.3 Crypto Stack Architecture Overview 12
Figure 2-3 Architecture of the vHsm 13
Figure 2-4 Job processing flow overview 14
Figure 2-5 Startup Sequence Overview 15
Figure 3-1 Ramloop handling 20
Figure 3-2 Job processing inside interrupt service routine 21

Tables

Table 1-1 Component history 10
Table 3-1 Supported AUTOSAR standard conform features 16
Table 3-2 Not supported AUTOSAR standard conform features 16
Table 3-3 Features provided beyond the AUTOSAR standard 16
Table 3-4 Custom Key Elements 22
Table 3-5 Key Element Values 23
Table 3-6 Custom Crypto_AlgorithmFamilyType used by the driver 23
Table 3-7 Custom Crypto_AlgorithmModeType used by the driver 23
Table 3-8 Structure of the segment list for secure boot update 29
Table 3-9 Input and output data for vHsm Firmware Update 29
Table 4-1 Embedded Implementation 32
Table 5-1 vHsm_Core_ProcessJob 33
Table 5-2 vHsm_Core_CancelJob 34
Table 5-3 vHsm_Core_KeyElementSet 35
Table 5-4 vHsm_Core_KeyElementGet 35
Table 5-5 vHsm_Api_KeyValidSetAndPersist 36
Table 5-6 vHsm_Api_Local_KeyElementGet 37
Table 5-7 vHsm_Api_InitializeTempJob 37
Table 5-8 vHsm_Api_ProcessTempJob 38
Table 5-9 vHsm_Api_CancelTempJob 38
Table 5-10 vHsm_Api_KeyElementGetForCryptoJobs 39
Table 5-11 vHsm_Api_KeyElementSet 40
Table 5-12 vHsm_Api_KeyElementSet_Unchecked 40
Table 5-13 vHsm_Api_DisallowKeyUsage 41
Table 5-14 vHsm_Api_AllowKeyUsage 42
Table 5-15 vHsm_Init 42
Table 5-16 vHsm_TriggerSoftwareReset 42
Table 5-17 vHsm_TriggerSoftwareResetWithIpcClearing 43
Table 5-18 vHsm_WriteErrorLog 43
Table 5-19 vHsm_ExternalIsr 44
Table 5-20 vHsm_ExternalIsr 45
Table 5-21 vHsm_Api_Ramloop_PrepareExecution 45
Table 5-22 vHsm_Api_Ramloop_Exit 46
Table 5-23 Services used by the vHsm_Core 47
Table 5-24 vHsm_StartupCallout 47
Table 5-25 vHsm_SystemInit_Callout 48
Table 5-26 vHsm_FirstStartup_Callout 48
Table 5-27 vHsm_InvalidBlockDetected_Callout 49
Table 5-28 vHsm_InitOne_Callout 49
Table 5-29 vHsm_InitTwo_Callout 50
Table 5-30 vHsm_InitThree_Callout 50

Table 5-31 vHsm_NvM_ProcessBlock_Callout 51
Table 5-32 vHsm_NvMainFunctions_Callout 51
Table 5-33 vHsm_MainFunctionCyclic_Callout 52
Table 5-34 vHsm_RamLoop_Callout 52
Table 5-35 vHsm_PreRamLoop_Callout 53
Table 5-36 vHsm_FirmwareUpdate_GetMacKey_Callout 53
Table 5-37 vHsm_FirmwareUpdate_GetCipherKey_Callout 54
Table 5-38 vHsm_Custom_IsSecureBootAllowAutostart_Callout 54
Table 5-39 vHsm_Custom_IsSecureBootAllowCyclicVerification_Callout 55
Table 5-40 vHsm_Custom_SecureBootSanction_Callout 55
Table 5-41 vHsm_SecureBoot_SegmentSuccess_Callout 56
Table 5-42 vHsm_Log_Event_Callout 56
Table 5-43 vHsm_Log_FillLevelThreshold_Callout 57
Table 5-44 vHsm_Log_GetTimestamp_Callout 57
Table 7-1 Glossary 59
Table 7-2 Abbreviations 59

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
1.00.00	Initial Creation
1.01.00	Feature Completion
1.02.00	- Preconfigured key storage concept - Support random initialization of key elements
2.00.00	- Support for IPC Protocol version 2
2.01.00	- Improved secure boot concept - Changes in static NVM layout configuration can be detected to reset the key storage - Added service for repersisting the whole key storage - Added support for Crypto_KeyElementCopyPartial API - Support redirection of input/output buffers from/to key elements - Added vHsm Firmware Update Service - CMAC processing optimization - Forwarding of all KeyM functions to configured drivers
2.01.01	Add restriction for Jump to Updater
2.01.02	Add internal only keys
3.00.00	- Use NvStorage handling of Crypto_30_LibCv and AUTOSAR NvM - Add Csm module to vHsm - Support multiple custom driver - Use KeyId of CryIf instead of LibCv as key identifier
3.00.01	Add vHsm_NvM_InvalidBlockDetected_Callout()
3.01.00	- Add cyclic secure boot verification - Add event log
4.00.00	Add OS support
5.00.00	- Add support for allow/disallow key usage API - Add Prepare/Exit ram loop API - Use Csm jobs inside vHsm - Store secure boot information directly with NvM - Support Hash for Secure Boot - Remove Write-Once Key - Add vHsm_PreRamLoop_Callout - Adapt description of vHsm_RamLoopCallout (add return value)

Table 1-1 Component history

2 Introduction

This document describes the functionality, API and configuration of the module vHsm_Core. The vHsm Core provides all platform independent features of the vHsm Firmware.

2.1 Architecture Overview

The following figure shows where the vHsm_Core is located in the AUTOSAR architecture.

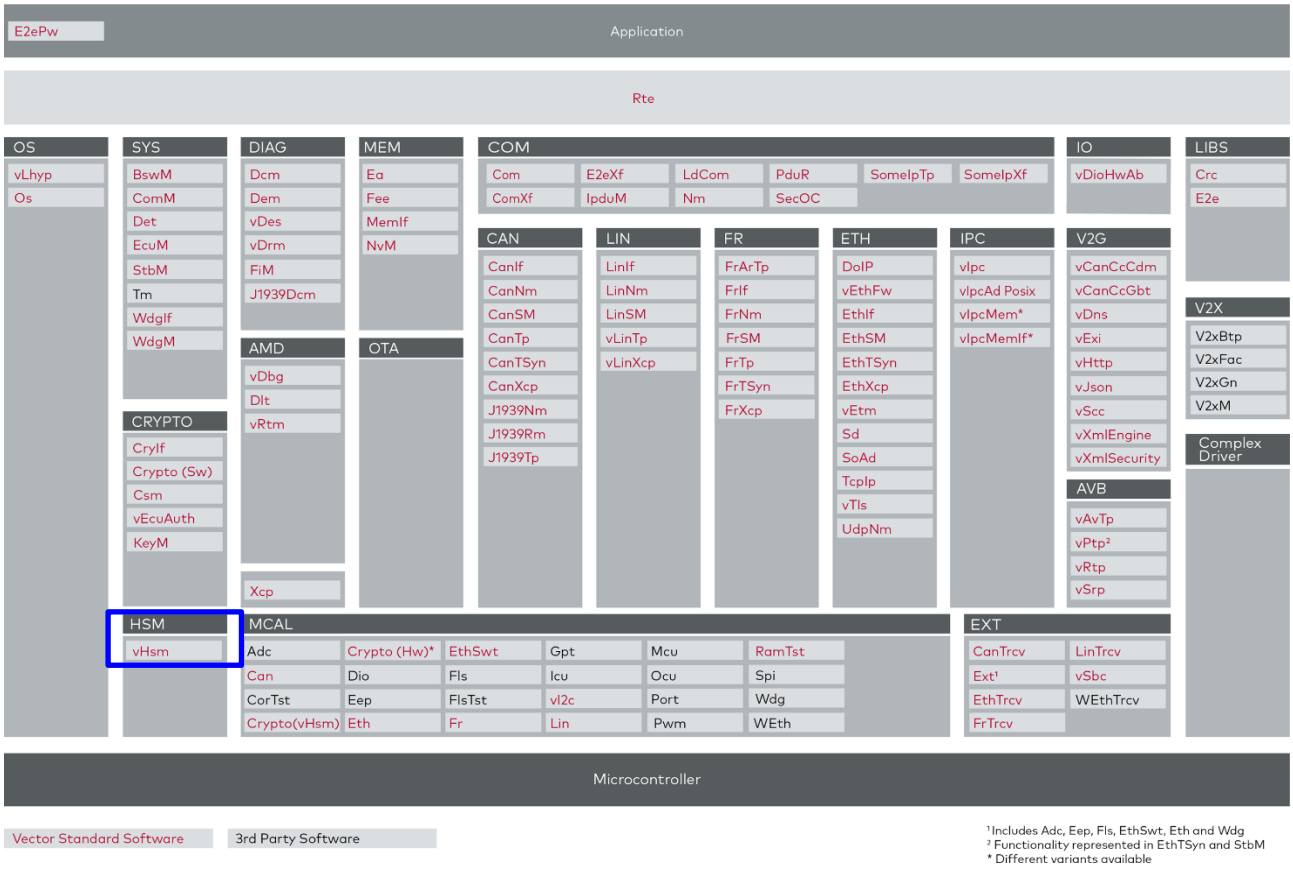


Figure 2-1 AUTOSAR 4.3 Architecture Overview

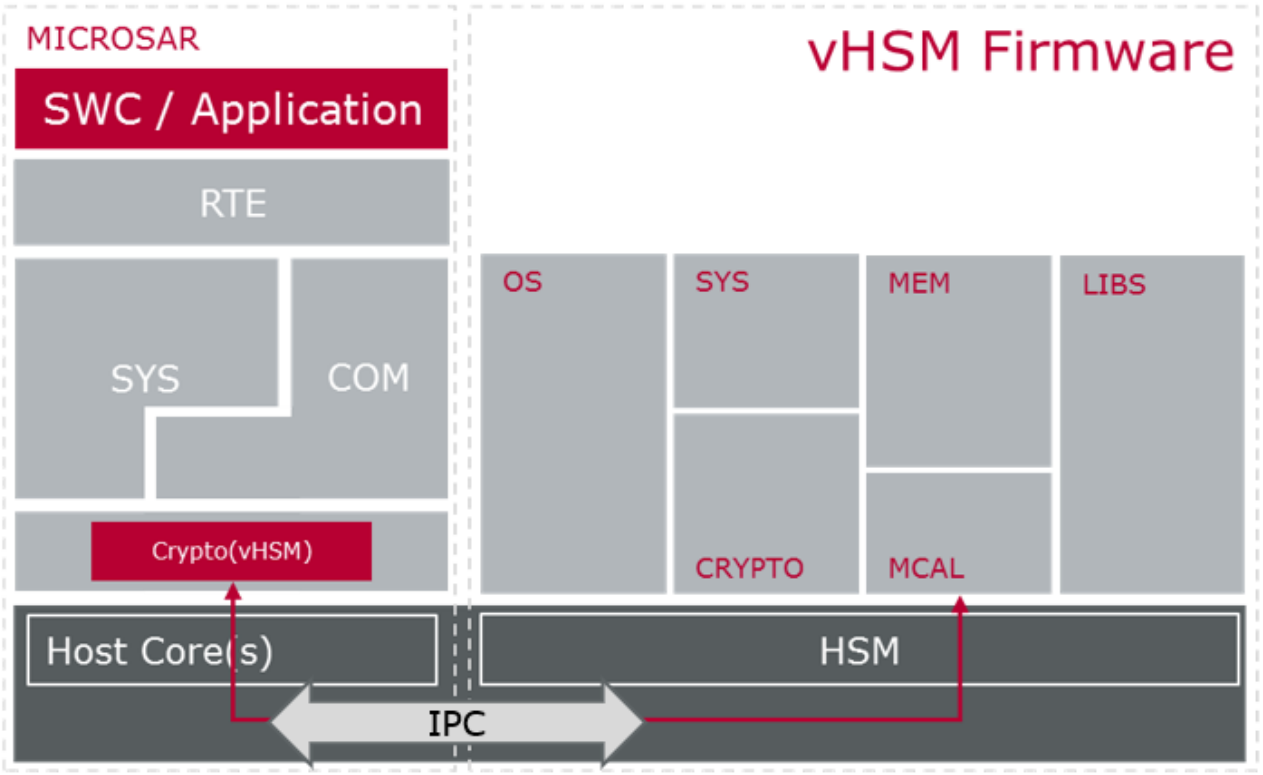


Figure 2-2 AUTOSAR 4.3 Crypto Stack Architecture Overview

2.2 vHsm Architecture Overview

The vHsm makes use of different AUTOSAR modules already available.

- CRYPTO drivers
  - CRYPTO vHsm_Core
    - Secure boot
    - Secure boot update
    - Firmware update
    - Software download
  - CRYPTO LibCv
    - Algorithms implemented in software
    - Key storage (including persistence via NvM)
  - CRYPTO vHsm_Hal provides hardware accelerated algorithms (e.g. CMAC)
  - CRYPTO vHsm_Custom provides customer specific functionalities
  - CRYPTO KeyM provides functionality of the KeyM
- CRYIF is used to dispatch jobs to different CRYPTO drivers
- NvM, FEE and vMem/FLS (on some plattforms from MCAL) provide the functionality to read and write to the data flash

© 2020 Vector Informatik GmbH

- vSecPrim provides the low-level implementation of the algorithms used by the CRYPTO drivers
- OS for scheduling of tasks
- RTE
- KeyM provides certificate handling
- CRYPTO KeyM

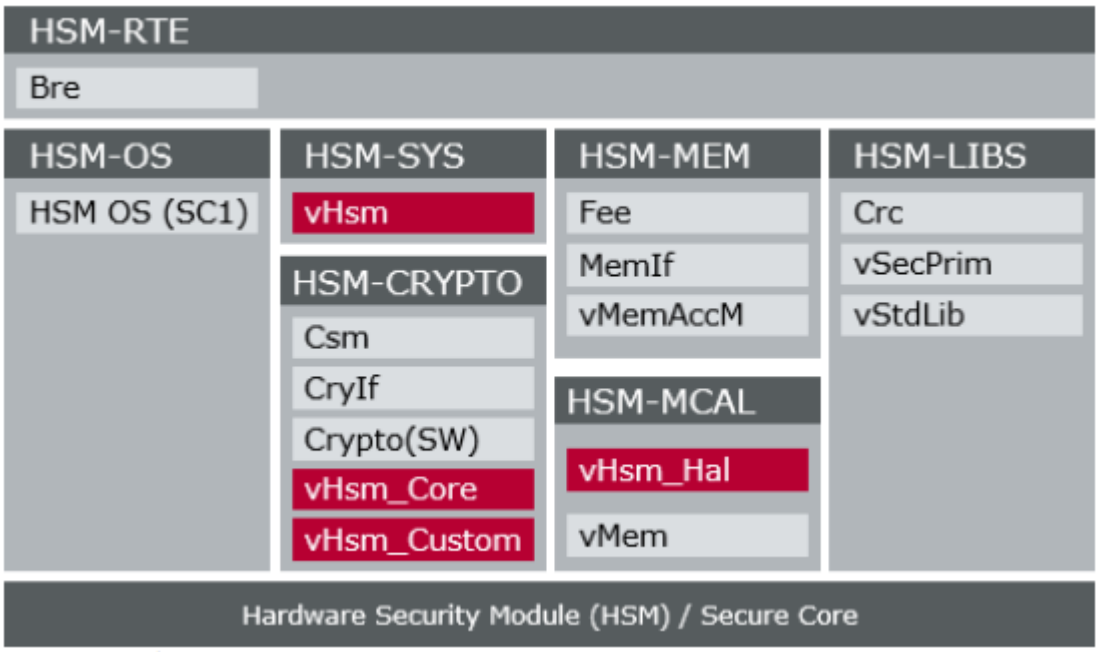


Figure 2-3 Architecture of the vHsm

2.3 vHsm Job Processing Overview

The vHsm on the HSM core receives jobs over the IPC from the CRYPTO driver on the application side. These jobs can be either jobs to process an algorithm (e.g. RSA, AES, CMAC, …) or a Key Management function (e.g. KeyElementSet, KeySetValid, …).

These Jobs are processed and sent to the corresponding CRYPTO drivers inside the vHsm.

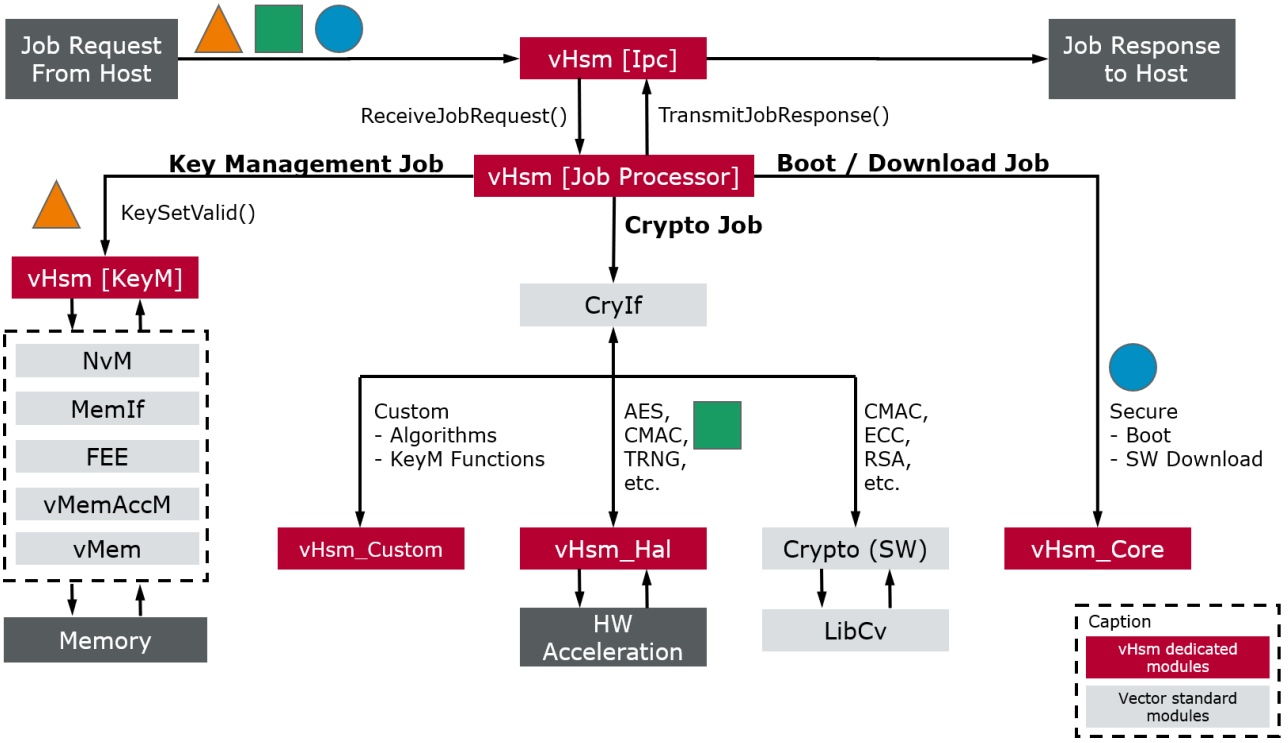


Figure 2-4 Job processing flow overview

2.4 vHsm Startup Sequence Overview

The startup sequence inside and the working loop of the vHsm is described in Figure 2-5

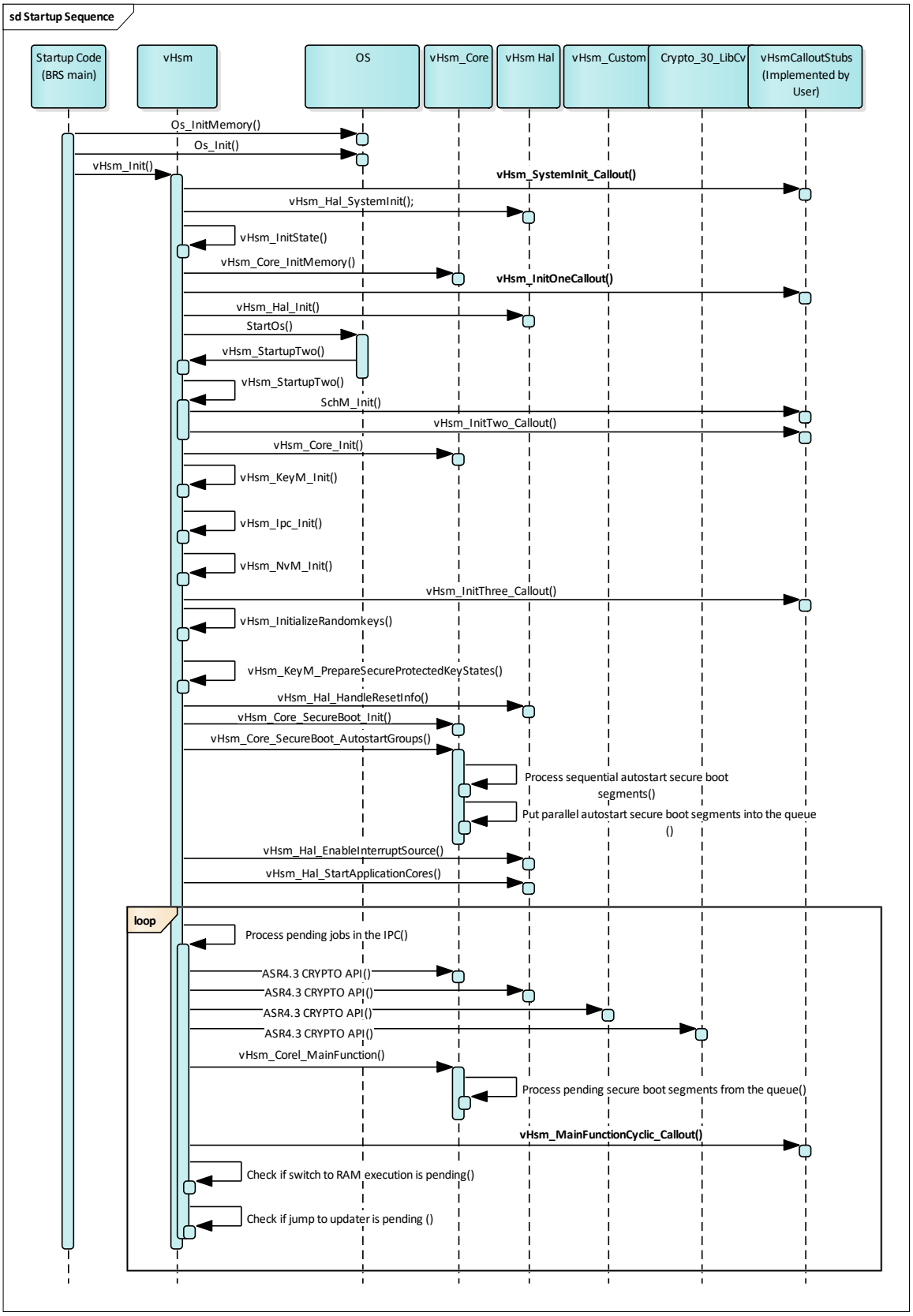


Figure 2-5 Startup Sequence Overview

3 Functional Description

3.1 Features

The vHsm_Core consists of the basic logic for the vHsm Firmware and a part which is API compatible to the AUTOSAR CRYPTO driver which is specified in [6].

The features listed in the following tables cover the complete functionality specified for the vHsm_Core.

The AUTOSAR standard functionality is specified in [2], the corresponding features are listed in the tables

> Table 3-1 Supported AUTOSAR standard conform features

> Table 3-2 Not supported AUTOSAR standard conform features

Vector Informatik provides further vHsm_Core functionality beyond the AUTOSAR standard. The corresponding features are listed in the table

> Table 3-3 Features provided beyond the AUTOSAR standard

The following features specified in [2] are supported.

Supported AUTOSAR Standard Conform Features
Only custom algorithms

Table 3-1 Supported AUTOSAR standard conform features

3.1.1 Deviations

The following features specified in [2] are not supported:

Not Supported AUTOSAR Standard Conform Features
All algorithms not stated in Table 3-1
Only KeyElementSet and KeyElementGet are implemented by this Crypto driver. All other Key management functions are forwarded to vHsm_Core module.
API function are prefixed with **vHsm_Core_** instead of **Crypto_**. Compatibility defines are provided with the module implementation.
There is no support for asynchronous jobs, all crypto jobs are handled synchronously.

Table 3-2 Not supported AUTOSAR standard conform features

3.1.2 Additions/ Extensions

The following features are provided beyond the AUTOSAR standard:

Features Provided Beyond The AUTOSAR Standard
Use vStdLib for memory access

Table 3-3 Features provided beyond the AUTOSAR standard

3.2 Initialization

The function `vHsm_Init()` will take care of the whole initialization of the modules used by the vHsm and acts as starting point for the firmware.

3.3 Error Handling

3.3.1 Development Error Reporting

By default, development errors are reported to the DET using the service `Det_ReportError()` as specified in [3], if development error reporting is enabled (i.e. pre-compile parameter `vHsm_Core_DEV_ERROR_REPORT==STD_ON`).

The occurrence of an error can be forwarded over the IPC to the application core.

3.4 vHsm

3.4.1 Pre Config File

The vHsm has the capability to generate a pre-config with the current settings of vHsm needed by CRYPTO, i.e. configured keys or driver objects including primitives.

This can be done for each IPC instance in the vHsm. The folder containing the pre-config file can be added to the list of additional definitions under the Project Settings of the application configuration.



Note
This Pre-Config mechanism may not work when a new driver is used with an old firmware version or vice versa.

3.4.2 Communication Registers

Most Platforms have at least two registers which can be used to share information between the application and HSM core. Otherwise shared RAM can be used.

3.4.2.1 HSM2HOST

The vHsm provides the internal state to the application core with HSM2HOST.

- Bit 0 is set at startup of the vHsm
- Bit 1 is set when the vHsm starts the application cores and is ready to receive commands and interrupts from the application core

3.4.2.2 HOST2HSM

The vHsm expects the application core to initialize the IPC. An initialized IPC memory is indicated by writing 1 to HOST2HSM. The bit position corresponds to the IPC instance Id.



Note
The Crypro_vHsm driver is initializing the IPC and indicating it via HOST2HSM since version 2.03.00.

If an older driver is used:

- the application must be adapted to init the IPC and indicate it to the HSM
- or the IPC needs to be initialized at startup inside the vHsm and `vHsm_State.vHsm_IpcInitialized[ipc]` needs to be set to `TRUE`.



Caution
If the IPC is not marked as initialized, the vHsm will not fetch jobs from the shared memory.

3.4.3 IPC Versions

The vHsm_Core and the vHsm communicate over a RAM based IPC protocol. However, this protocol has a version which needs to be the same on both sides.

For HSM versions starting from Protocol Version 2, the version is synchronized via the generated pre-config file described in chapter 3.4.1. This pre-config file should be added as additional defines in the CRYPTO vHsm driver.

For earlier versions, set this parameter manually to 1.

Protocol Version 1 only allows the usage of 4 driver objects of the vHsm:

- **Core:** Can be used to issue the secure boot and secure software update jobs
- **LibCv:** Can be used to compute algorithms inside the software implementation on the vHsm. The available primitives depend on the vHsm configuration and can be synchronized with a pre-config file generated by the vHsm.
- **Hal:** Can be used to compute algorithms in hardware such as CMAC, AES and TRNG
- **KeyM:** Cannot be used to perform jobs. This driver object is internally used to send the key management API calls to the vHsm. Do not reference this driver object in the CRYIF.

With Protocol Version 2 and higher, there is no limit for driver objects anymore.



Note
It is highly recommended to use IPC Version 2

3.4.4 Memory Access

The input and output data must be accessible by the HSM Core. A whitelist of accessible memory areas can be configured in the vHsm. If a pointer is not inside this whitelist, the job will be accepted and the vHsm Firmware returns the job with E_NOT_OK.



Caution
Be careful to not include memory areas of the vHsm (e.g. code, const and var data of the firmware) into this list. This may allow an adversary to read contents of vHsm secure RAM or secure data flash. For example, an adversary could encrypt contents of the secure RAM with a known key and decrypt the received memory contents again.



Caution
If the vHsm shall be used in a safety use-case, it must be ensured that the HSM can only modify allowed memory areas (memory sections that are configured to be writable from HSM). This can be achieved with an MPU. Refer to the corresponding hardware manual of the platform.
The driver on the host core can be configured to always provide output pointers pointing to the vHsmGlobalRamBuffer (memory section that is write and readable from the vHSM). The HSM can therefore be restricted to only be able to have write access to vHsmIpcMemory and vHsmGlobalRamBuffer.

3.4.5 Key Storage

The vHsm uses the CRYPTO driver Crypto_30_LibCv as key storage. Persisting is done with trough the AUTOSAR memory stack consisting of NvM, Fee and Fls/vMem. The keys configured in the Crypto_30_LibCv will be available for use by the vHsm and all its included modules through the services provided in vHsm_Api.h described in chapter 5.2.

Loading the keys from data flash to the RAM will be performed during startup of the vHsm. All keys are cached in RAM and therefore do not need to be loaded on demand from the data flash.

Key elements which have been marked to be persisted can either be stored immediately when key is set to valid or deferred through an extra API call described in chapter 3.4.10.5.

3.4.6 Interrupts

The vHsm notifies the application core about a completed operation with the response interrupt. The interrupt is always triggered.

The application does not necessarily need to have the response interrupt enabled as the driver can also run in polling mode which does not need an ISR to be called. Instead, the driver will poll the IPC memory for available responses from the vHsm Firmware.

It is also possible that the application core triggers interrupts when issuing a job request to the vHsm. This is needed if OS is used or jobs shall be processed directly in the interrupt context.

3.4.7 Ramloop

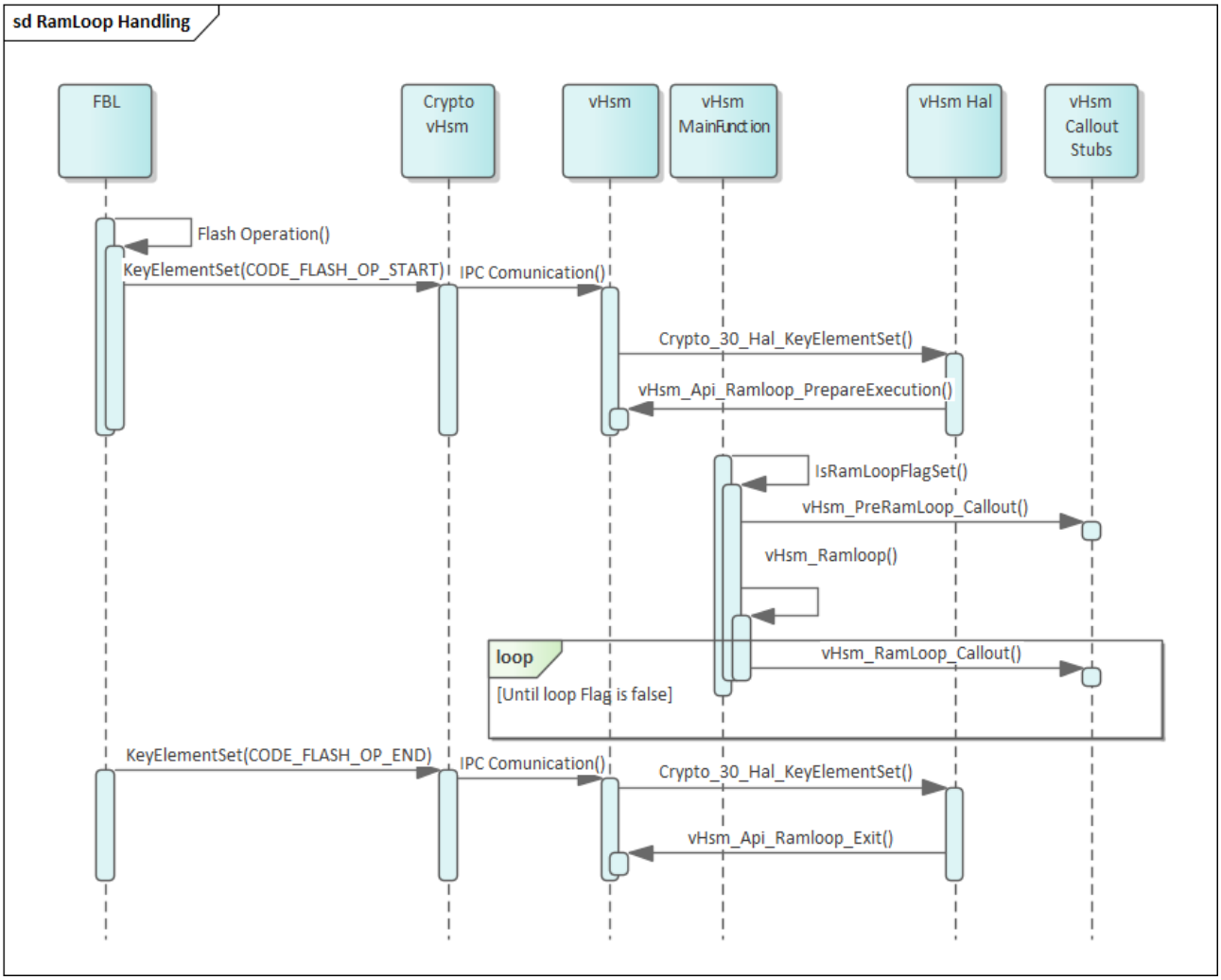


Figure 3-1 Ramloop handling

On some platforms it is necessary that the vHsm runs in RAM while a codeflash access is performed. How the vHsm can be sent to RAM execution is described in [3].

Figure 3-1 depicts the sequence of the ramloop handling. The relevant API calls are described in section 5.2, the callouts in 5.4.1.

3.4.8 Multiple tasks

It is possible to have multiple tasks running on the vHsm to be able to process time critical jobs while a long running job will be interrupted.

Caution
Having multiple tasks always requires a Driver on the application core which can trigger a request interrupt to the vHsm Firmware when a job request is performed

3.4.8.1 Processing inside ISR

On resource restricted platforms or where fast execution time matters, jobs can be processed directly in the ISR. When a job shall be executed on a CryIfChannel referenced under `/MICROSAR/vHsm/vHsmScheduling/vHsmIsrProcessedChannel`, the processing will be inside the ISR. This way, long running jobs (e.g. Signature Verify) in another context will be interrupted.

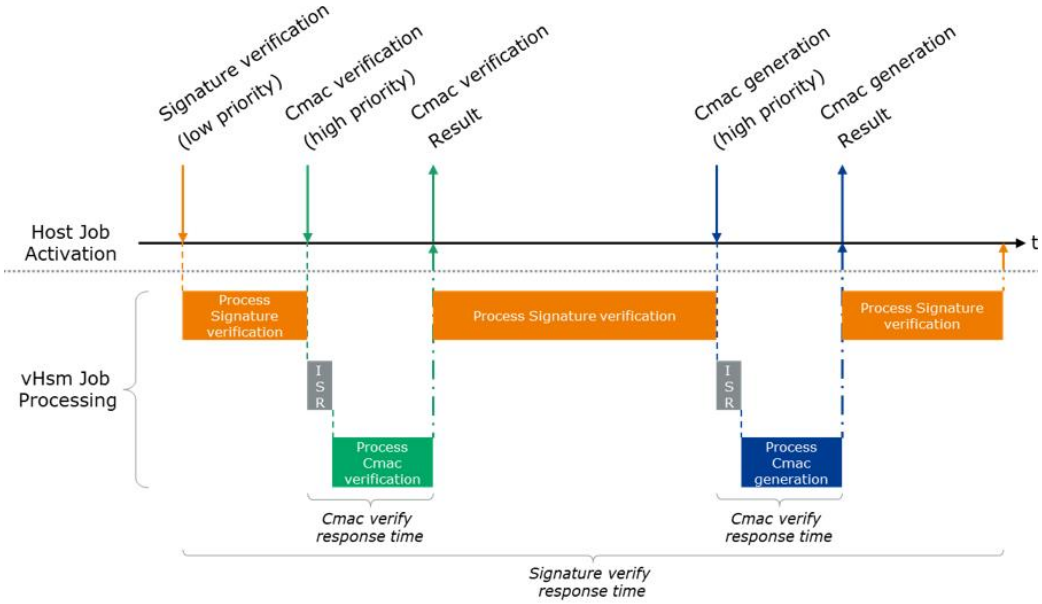


Figure 3-2 Job processing inside interrupt service routine

3.4.8.2 Os

The vHsm supports usage of an MICROSAR OS for scheduling. This enables the user to process jobs on different tasks with different or same priorities.

One or more driver objects can be grouped to a schedulable entity which can then be mapped to a task. The task itself is then automatically generated by the RTE.

If a job is received on one of those driver objects, the corresponding task is activated, and processing starts.

Note
This can be used to have low running signature jobs running on a low prio task while other jobs like CMAC can be processed with a higher priority which will interrupt the signature jobs.

3.4.9 Custom definitions

3.4.9.1 Custom Key Elements

There are custom key elements defined, which are shown in the following table and are used for custom services described in chapter 3.4.10.

Custom Key Element	Value
CRYPTO_KE_CUSTOM_VHSM_VERSION	3020
CRYPTO_KE_CUSTOM_VHSM_UID	3021
CRYPTO_KE_CUSTOM_VHSM_PERFORM_PERSISTING	3022
CRYPTO_KE_CUSTOM_VHSM_ERRORLOG	3023
CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_SLOT_ADDRESS	3024
CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_SLOT_SIZE	3025
CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_SLOT_CMAC	3026
CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_SLOT_TAG	3026
CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_SLOT_KEY	1
CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_SLOT_SANCTION	3027
CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_END	3028
CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_RESULT	3037
CRYPTO_KE_CUSTOM_VHSM_PERFORM_REPERSISTING	3036
CRYPTO_KE_CUSTOM_VHSM_FIRMWARE_UPDATE_START_UPDATER	3038
CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_SLOT_MODE	3039

Table 3-4 Custom Key Elements

3.4.9.2 Custom Key Element Values

Some key elements only accept specific values, which are shown in the following table and are used for custom services described in chapter 3.4.10.

Key Element	Define	Value
CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_SLOT_SANCTION	CRYPTO_30_VHSM_SANCTION_NONE	0x00
	CRYPTO_30_VHSM_SANCTION_RESET	0x01
	CRYPTO_30_VHSM_SANCTION_CUSTOM	0x02
	CRYPTO_30_VHSM_SANCTION_HALT	0x03
CRYPTO_KE_CUSTOM_VHSM_SECURE	CRYPTO_30_VHSM_MODE_NONE	0x00

	CRYPTO_30_VHSM_MODE_PARALLEL	0x01
	CRYPTO_30_VHSM_MODE_SEQUENTIAL	0x02

Table 3-5 Key Element Values

3.4.9.3 Custom Algorithm Family

There are custom algorithm families defined, which are shown in the following table and are used for custom services described in chapter 3.4.10.

ID	Define	Service	Description
160	CRYPTO_ALGOFAM_CUSTOM_VHSM_SECURE_BOOT	CRYPTO_MACVERIFY	Used for triggering the verification of secure boot slots
161	CRYPTO_ALGOFAM_CUSTOM_VHSM_SECURE_SOFTWARE_DOWNLOAD	CRYPTO_SIGNATUREVERIFY	Used for performing a secure software download to update the contents of a secure boot slot
163	CRYPTO_ALGOFAM_CUSTOM_VHSM_FIRMWARE_UPDATE	CRYPTO_AEADENCRYPT	Used for performing the firmware update service.
164	CRYPTO_ALGOFAM_CUSTOM_VHSM_SECURE_BOOT_UPDATE	CRYPTO_SIGNATUREVERIFY	Used for performing a secure boot update to update secure boot groups using signed a segment list

Table 3-6 Custom Crypto_AlgorithmFamilyType used by the driver

3.4.9.4 Custom Algorithm Mode

There are custom algorithm modes defined, which are shown in the following table and are used for custom services described in chapter 3.4.10.

ID	Define	Service	Description
133	CRYPTO_ALGOMODE_CUSTOM_SECURE_BOOT_FORCE_SEQUENTIAL	CRYPTO_MACVERIFY	Used for triggering the verification of a secure boot group/segment and enforce the sequential processing and thus avoid putting segments to the queue.

Table 3-7 Custom Crypto_AlgorithmModeType used by the driver

3.4.10 Custom Services

The communication with vHsm is done only with CRYPTO ASR4.3 compatible APIs. Custom behavior can be achieved by using custom keys and custom elements in combination with `KeyElementSet()` and `KeyElementGet()`. The vHsm catches this special keys and key element ids and performs the behavior described below instead of simply setting / reading a key. All other keys and elements are handled as usual.

3.4.10.1 Version Info

Call `vHsm_Core_KeyElementGet()` to retrieve the key element `CRYPTO_KE_CUSTOM_VHSM_VERSION` from the info key. The version consists of three bytes which are the concatenation of `MAJOR|MINOR|BUGFIX`.

3.4.10.2 UID

Call the function `vHsm_Core_KeyElementGet()` to retrieve the key element `CRYPTO_KE_CUSTOM_VHSM_UID` from the info key. The info key is the key which is referenced under `/vHsm/vHsmKeyMapping/vHsmInfoKey` in the vHsm configuration.

3.4.10.3 Error Log

Call the function `vHsm_Core_KeyElementGet()` to retrieve the key element `CRYPTO_KE_CUSTOM_VHSM_ERRORLOG` from the info key.

The latest error will be written to the beginning. An error entry consists of two uint32 values (errorId followed by the errorData). Older errors will be moved back until there is no more space in the key available, which will discard the oldest log entry.

The error log is only active when `vHsm/vHsmGeneral/ErrorLogEventLogRef` is set.



Expert Knowledge
The data of the event log will not be stored in key elements anymore. Instead dedicated memory is allocated, and persisting is done directly with the NvM.

3.4.10.4 Event Log

Call the function `vHsm_Core_KeyElementGet()` to retrieve the key `vHsmLog` from the core driver. The key element Id will be interpreted as LogEventId.

Depending on the configured export strategy, the data of the event will be written to the output.

3.4.10.5 Lazy Persist Keys

Keys which are referenced in `CryptoNvBlocks` inside the LibCv which are set to `NV_PROCESSING_DEFERRED` will not be persisted when they are set to valid. The key element `CRYPTO_KE_CUSTOM_VHSM_PERFORM_PERSISTING` must be set to any value inside the info key to trigger the persisting of the lazy keys.

For more information refer to the documentation of the LibCv driver.

3.4.10.6 Write Once Keys

Write Once Keys are used for Keys which can only be written once and are blocked for further write operations.

3.4.10.7 Repersist Keys

On some platforms the manufacturer requires the data flash to be rewritten after a defined period to guarantee the retention times.

Repersisting of the data flash can be triggered from the application by setting the key element `CRYPTO_KE_CUSTOM_VHSM_PERFORM_REPERSISTING` to any value inside the info key.

3.4.10.8 Secure Boot

3.4.10.8.1 Storage Method

There are two possibilities to store the information (e.g. address, size, tag, …) of secure boot segments and groups:

- **Storage in NvM** will use the NvM directly. Advantages are less overhead runtime overhead, RAM and ROM consumption. Segments can be combined to store a tag per group or a tag per processing to reduce memory consumption even more.
- **Storage in Csm** (deprecated) will use the KeyStorage provided by the LibCv driver for persisting the secure boot information. Each segment always has an own tag.



Caution
Storage in Csm is deprecated and will be removed in future versions

3.4.10.8.2 Verification Method

The vHsm supports MAC (CMAC) as well as HASH (e.g. SHA-256, SHA-512, …) for protecting against changes in memory.

Each group can have a distinct verification method. The Csm jobs referenced in the verification method define the algorithm and which key is used.



Expert Knowledge
If it is necessary to use different keys for each segment, a dedicated group for each segment needs to be created. Then it is possible to reference different keys in the corresponding Csm jobs.

3.4.10.8.3 Secure Boot Segment

This is a memory area which will be secured with a tag.

A segment has three modes:

- **Sequential** segments are immediately processed. The vHsm is busy until the segment is finished
- **Parallel** segments will be added to the secure boot queue which will be processed after startup during the main loop in small parts. Processing of other jobs from the application is possible, but not with the driver object used for the parallel jobs.
- **None** segments will not be processed

The tag of the secure boot segment will be filled during the first verification. All subsequent verifications will be checked against this stored tag.

In the configuration the initial values (e.g. address, size, mode, sanction) can be set for each segment.

3.4.10.8.4 Secure Boot Group

A secure boot group consist of multiple segments. The groups can be used to define e.g. an application area, divided into smaller parts because of gaps of unwritten code flash in the memory layout.

3.4.10.8.5 Automatic start of secure boot

Secure boot groups can be configured to be automatically started.

The automatic start can be restricted with a callout which can be implemented by the user if needed.



Expert Knowledge
The custom callout is for restricting the automatic start and can e.g. be used to decide if a group shall be started depending on the current reset vector configuration.

When a group verification is started, all sequential segments of the group will immediately be processed. This will block the startup and may lead to long startup times of the vHsm Firmware depending on the checked memory sizes.

All parallel segments will be added to the secure boot queue and will be processed later during the normal operation of the vHsm.

3.4.10.8.6 Cyclic Secure Boot Verification

When secure boot has finished, the cyclic groups will be re-added to the processing queue one after another as soon as the queue is empty.



Expert Knowledge
The custom callout is for restricting the cyclic start and can e.g. be used to decide if a group shall be started depending on e.g. the value of a key element or another state.

3.4.10.8.7 Manual trigger of secure boot groups and segments

Call a MAC verify job referencing a primitive which has the algorithm family set to `CRYPTO_ALGOFAM_CUSTOM_VHSM_SECURE_BOOT`.

There are two ways to decide which group or segment shall be verified:

- When dataPtr points to a uint32 variable and dataLength is 4, the passed data will be interpreted as GroupId to select the corresponding group. (The keyId referenced in the job will be ignored in this case)
- (deprecated!) When the variable `dataLength` is 0, the key referenced in the job is used to select the appropriate group or segment.

The parameters macPtr and macLength are ignored and can be set to NULL_PTR and 0.

The variable pointed by `verifyPtr` will be filled with the result of the sequentially checked segments.

To force the processing to be done sequentially disregarding the fact that a segment may be currently configured as being processed as parallel secure boot, use

CRYPTO_ALGOMODE_CUSTOM_SECURE_BOOT_FORCE_SEQUENTIAL as mode of the job primitive or enable the configuration parameter vHsmSecureBootForcedSequentialProcessingOnFirstVerification.



Caution
One some platforms, concurrent data flash access from application core and HSM is not possible. Depending on the synchronization strategy it may be necessary to force the sequential processing of parallel segments to ensure that tags are written to the data flash. While forcing those segments, the HSM needs to have the right to access the data flash.

Triggering the forced sequential segment verifications can be done in the callout stubs, from the application core or by enabling vHsmSecureBootForcedSequentialProcessingOnFirstVerification in the configuration



Expert Knowledge
The sanction “Change reset vectors” has been removed and replaced by a custom callout. This can be used to change the page in an A/B Concept when e.g. page A has been detected as invalid.



Caution
Secure Boot cannot be done from address 0x00000000.

If flash starts on 0x00000000, skip first page size, e.g. 256 bytes, and set reset vector and secure boot slot accordingly.

This is because AUTOSAR specifies NULL_PTR checks inside the whole stack which prevents using 0x00000000 as legitimate address.

3.4.10.8.8 Indicate the end of Secure Boot

To indicate the end of secure boot and disallow verifying new groups or segments, call KeyElementSet() with the vHsm_Info key and the key element set to CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_END. When all unfinished parallel segments have been processed, secure boot protected keys may get usable again depending on the result.

If secure boot is finished can be checked by calling KeyElementGet with the vHsm_Info key and the keyElementId set to CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_END. When no error is reported, the first byte will be 1 if secure boot is finished.

3.4.10.8.9 Secure boot results

To get the overall secure boot result after a finished secure boot, call KeyElementGet() with the vHsm_Info key and the key element

CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_RESULT. When no error is returned, the first byte in the buffer will contain the result of the secure boot.

To check the current state (secure boot does not need to be finished at this point) of a secure boot group, call KeyElementGet() with the vHsm_Info_SecureBootGroup key and set the keyElementId to the Id of the group configured in the DaVinci Cfg5. When no error is returned, the first byte of the provided buffer will indicate if the group is finished (no current processing segments) and the second byte will contain the current result of the group.

3.4.10.9 Secure Software Download

To perform a secure software download, which means updating an already stored tag in a single secure boot segment, call a signature verify job, which has the algorithm family set to CRYPTO_ALGOFAM_CUSTOM_SOFTWARE_DOWNLOAD.

The provided signature will be validated against the key which is referenced in vHsm/vHsmSecureBoot/vHsmSecureSoftwareDownload/vHsmSignatureKey inside the vHsm Configuration. If the signature over the provided data is valid, the tag, address and size of the secure boot slot will be updated.

The variable pointed by verifyPtr will be filled with the result of the signature verification.



Note
The updated secure boot segment is only stored to the key storage in the RAM. Call KeyValidSet() to persist the secure boot slot to the NVM after a successful secure software download.



Expert Knowledge
The secure software download can be used in the streaming approach. This means that multiple UPDATE calls can be performed to provide the data in separate parts.

The underlying primitive for the signature may not be able to process more than 64k of data at once. Consider using the streaming in this case.

3.4.10.10 vHsm Secure Boot Update

The secure boot update is used to update all segments inside a group at once. To perform a secure boot update, call a signature verify job with the algorithm family set to CRYPTO_ALGOFAM_CUSTOM_VHSM_SECURE_BOOT_UPDATE.

The dataPtr contains the segment list and a valid signature over the segment list needs to be provided. The structure of the segment list is described in Table 3-8. The service can only be used as SINGLECALL and does not support streaming.

If the signature over the segment list is valid, the Hash of each segment will be checked. When successful, the tag for this segment will be computed and the updated address, length, mode and tag will be stored to the key storage. When the segment list contains

less segments than the group, the remaining segments of the group will have the mode set to None and thus will not be used for secure boot.

When all segments of the segment list have been updated successfully, the corresponding segments will be persisted.

Name	Size (in Bytes)	Description
GroupId	4	Id of the Group which must match a group inside the configuration
n Segments	2	Number of segments provided inside this segment list
Address	4	Address of segment #0
Length	4	Length of segment #0
Mode	2	Mode of segment #0 (e.g. Parallel, Sequential, None)
Hash	32	SHA256 over segment #0
…		
Address	4	Address of segment #n-1
Length	4	Length of segment #n-1
Mode	2	Mode of segment #n-1
Hash	32	SHA256 over segment #n-1

Table 3-8 Structure of the segment list for secure boot update

3.4.10.11 vHsm Firmware Update

To perform a vHsm Firmware Update, call `vHsm_Core_ProcessJob()` with a AEAD Encrypt job, which has the algorithm family set to `CRYPTO_ALGOFAM_CUSTOM_VHSM_FIRMWARE_UPDATE` and mode to GCM. `START` must be performed separately. `UPDATE` can be called multiple times and can be combined with the `FINISH` at the end.

Mode	Parameter	Description
START	input	UpdContainerHeader
	secondary Input	Update header
UPDATE	input	Encrypted firmware image
	output	Re-crypted firmware image
FINISH	secondary input	Signature over UpdContainerHeader & encrypted firmware image
	secondary output	MAC over re-crypted firmware image and parts of the update header

Table 3-9 Input and output data for vHsm Firmware Update

After a successful firmware update, the vHsm can be instructed to jump to the updater by calling KeyElementSet with the vHsm_Info key and the key element set to `CRYPTO_KE_CUSTOM_VHSM_FIRMWARE_UPDATE_START_UPDATER`.

3.4.11 Limitations

3.4.11.1 Secure Boot

Secure Boot cannot be done from Address zero (NULL_PTR). This is because the AUTOSAR APIs specify checks against NULL_PTR at various locations inside the stack.

If flash starts on 0x00000000, skip first page size, e.g. 256 bytes, and set reset vector and secure boot slot accordingly.

3.4.11.2 Platforms with memory windows

On Platforms with memory windows (e.g. Tricore, Rcar) the IPC memory and the global RAM buffers of all IPC instances need to be placed in one window. This limitation can be ignored, when only one IPC instance is used.

4 Integration

This chapter gives necessary information for the integration of the MICROSAR vHsm_Core into an application environment of an ECU.

4.1 Embedded Implementation

The delivery of the vHsm_Core contains these source code files:

File Name	Description	Integration Tasks
Crypto_30_vHsm_Core.h	This header file is used to redirect function names generated by the CRYIF.	-
Det.h	This file contains the redirection of DET function calls to vHsm handler functions.	-
vHsm.c	This is the main source file of the vHsm_Core. It contains functions for initialization of the component, the main function and utility functions.	-
vHsm.h	This is the header file of the vHsm_Core. It contains the revision history of the component.	-
vHsm_Api.c	This file contains API functions definitions which can be used by the crypto drivers to minimize dependencies to the LibCv driver.	-
vHsm_Api.h	This file contains API functions declarations which can be used by the crypto drivers to minimize dependencies to the LibCv driver.	-
vHsm_Api_Internal.h	This file contains internal used API's to minimize dependencies to the LibCv driver.	-
vHsm_Core.c	This source file contains the implementation of the driver API which provides basic functionalities like secure boot or reading the version.	-
vHsm_Core.h	This header file defines the public interface of the Core crypto driver.	-
vHsm_Core_Custom.h	This header file contains custom key element definitions for custom algorithms.	-
vHsm_Core_SecureBoot.c	This is the source file for the secure boot job processing.	-
vHsm_Core_SecureBoot.h	This is the header file for the secure boot job processing.	-
vHsm_Det.c	This is the source file for the DET.	-
vHsm_Det.h	This header file contains definitions for the DET.	-
vHsm_Ipc.c	This is the source file for the IPC.	-
vHsm_Ipc.h	This is the header file for the IPC.	-

File Name	Description	Integration Tasks
vHsm_Ipc_Types.h	This header file contains types of the IPC.	-
vHsm_JobProcessor.c	This is the source file for the job processor.	-
vHsm_JobProcessor.h	This is the header file for the job processor	-
vHsm_Log.c	This is the source file for logging.	-
vHsm_Log.h	This is the header file for logging.	-
vHsm_KeyM.c	This is the source file for key management function handling.	-
vHsm_KeyM.h	This is the header file for key management function handling.	-
vHsm_Nvm.c	This is the source file for the persisting of the key storage.	-
vHsm_Nvm.h	This is the header file for the persisting of the key storage.	-
vHsm_Trace.c	This is the source file for debug output.	-
vHsm_Trace.h	This is the header file for debug output.	-
vHsm_Types.h	This header file contains types for the vHsm.	-
vHsm_Cfg.c	This is generated source of the component configuration.	-
vHsm_Cfg.h	This is generated header of the component configuration.	-
vHsm_Callout_Stubs.c	Generated **Template File** that must be adapted by the user to the system needs.	File can either be modified directly in the User blocks or can be excluded from the project so that callout definitions can be made in any other file.

Table 4-1 Embedded Implementation

4.2 Integration Hints

There is an additional technical reference manual in the delivery with platform specific documentation. This includes the memory configuration for the IPC and the description of Interrupts provided by the vHsm_Hal.

See [5] for more information.

5 API Description

5.1 Services provided by CRYPTO driver

All services of the ASR4.3 CRYPTO not described in the following subchapters are directly forwarded to the Crypto_30_LibCv driver.

5.1.1 vHsm_Core_ProcessJob

Prototype	
`Std_ReturnType` **`vHsm_Core_ProcessJob`** `(uint32 objectId, Crypto_JobType *job)`	
Parameter	
objectId [in]	Holds the identifier of the Crypto Driver Object.
job [in, out]	Pointer to the configuration of the job. Contains structures with job and primitive relevant information but also pointer to result buffers.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_NOT_VALID Request failed, the key is not valid.
	CRYPTO_E_QUEUE_FULL Request failed, the queue is full.
	CRYPTO_E_SMALL_BUFFER Request failed, the provided buffer is too small to store the result.
Functional Description	
Processes the received job. Performs the crypto primitive, that is configured in the job parameter.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-1 vHsm_Core_ProcessJob

5.1.2 vHsm_Core_CancelJob

Prototype	
`Std_ReturnType` **`vHsm_Core_CancelJob`** `(uint32 objectId, Crypto_JobType *job)`	
Parameter	
objectId [in]	Holds the identifier of the Crypto Driver Object.

job [in, out]	Pointer to the configuration of the job. Contains structures with user and primitive relevant information.
Return code	
Std_ReturnType	E_OK Request successful, job has been removed.
Std_ReturnType	E_NOT_OK Request failed, job could not be removed.
Functional Description	
Cancels the received job. This interface removes the provided job from the queue and cancels the processing of the job if possible.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-2 vHsm_Core_CancelJob

5.1.3 vHsm_Core_KeyElementSet

Prototype	
`Std_ReturnType` **`vHsm_Core_KeyElementSet`** `(uint32 cryptoKeyId, uint32 keyElementId, const uint8 *keyPtr, uint32 keyLength)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key whose key element shall be set.
keyElementId [in]	Holds the identifier of the key element which shall be set.
keyPtr [in]	Holds the pointer to the key data which shall be set as key element.
keyLength [in]	Contains the length of the key element in bytes.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_WRITE_FAIL Request failed, write access was denied.
	CRYPTO_E_KEY_NOT_AVAILABLE Request failed, the key is not available.
	CRYPTO_E_KEY_SIZE_MISMATCH Request failed, the key element size does not match size of provided data.
Functional Description	
Sets a key element. Sets the given key element bytes to the key identified by cryptoKeyId. .	
Particularities and Limitations	
-	
Call context	

> TASK > This function is Synchronous > This function is Reentrant

Table 5-3 vHsm_Core_KeyElementSet

5.1.4 vHsm_Core_KeyElementGet

Prototype	
`Std_ReturnType` **`vHsm_Core_KeyElementGet`** `(uint32 cryptoKeyId, uint32 keyElementId, uint8 *resultPtr, uint32 *resultLengthPtr)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key whose key element shall be set.
keyElementId [in]	Holds the identifier of the key element which shall be set.
resultPtr [in]	Holds the pointer to the key data which shall be set as key element.
resultLengthPtr [in]	Contains the length of the key element in bytes.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_READ_FAIL Request failed, read access was denied.
	CRYPTO_E_KEY_NOT_AVAILABLE Request failed, the key is not available.
	CRYPTO_E_KEY_SIZE_MISMATCH Request failed, the provided buffer is too small to store the result.
Functional Description	
This interface shall be used to get a key element of the key identified by the cryptoKeyId and store the key element in the memory location pointed by the result pointer.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-4 vHsm_Core_KeyElementGet

5.2 Services provided by vHsm_Core

5.2.1 vHsm_Api_KeyValidSetAndPersist

Prototype
`Std_ReturnType` **`vHsm_Api_KeyValidSetAndPersist`** `(uint32 cryptoKeyId)`

Parameter	
cryptoKeyId [in]	Holds the identifier of the key whose key elements shall be set to valid and persisted
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
Functional Description	
Sets the key to valid an persists the key if the valid set operation was successful. Sets the key state of the key identified by cryptoKeyId to valid and persists it to the NvM.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-5 vHsm_Api_KeyValidSetAndPersist

5.2.2 vHsm_Api_Local_KeyElementGet

Prototype	
`Std_ReturnType` **`vHsm_Api_Local_KeyElementGet`** `(uint32 cryptoKeyId, uint32 keyElementId, Crypto_30_LibCv_KeyStorageIndexType *resultIndexPtr, uint32 *resultLengthPtr)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key whose key element shall be provided.
keyElementId [in]	Holds the Id of the key element which shall be provided.
resultIndexPtr [in, out]	Holds the pointer to the storage index which shall be provided.
resultLengthPtr [in, out]	Contains the length of the key element in bytes.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_KEY_READ_FAIL Request failed, read access was denied.
	CRYPTO_E_KEY_NOT_VALID Request failed, the key is not valid.
	CRYPTO_E_KEY_NOT_AVAILABLE Request failed, the key is not available.
	CRYPTO_E_SMALL_BUFFER Request failed, the provided buffer is too small to store the result.
	CRYPTO_E_KEY_SIZE_MISMATCH Request failed, the provided buffer is too small to store the result.

Functional Description
The key element storage index can be provided for all read access rights values. This interface shall be used internally to get a key element of the key identified by the cryptoKeyId and store the key element in the memory location pointed by the result pointer. The key element will be searched and checked for validity.
Particularities and Limitations
cryptoKeyId must be a valid identifier to a key in the keyStorage. The crypto key must be locked for read access. resultIndexPtr must be a valid pointer. resultLengthPtr must be a valid pointer.
Call context
> TASK > This function is Synchronous > This function is Reentrant

Table 5-6 vHsm_Api_Local_KeyElementGet

5.2.3 vHsm_Api_InitializeTempJob

Prototype	
`void vHsm_Api_InitializeTempJob (vHsm_Api_TempJobType *tmpJob, uint32 cryptoKeyId)`	
Parameter	
tmpJob [in]	Pointer to the job which shall be initialized
cryptoKeyId [in]	Key which shall be used in the job
Return code	
void	none
Functional Description	
Generates a temporary job. The job can be modified to e.g. perform a AES calculation.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-7 vHsm_Api_InitializeTempJob

5.2.4 vHsm_Api_ProcessTempJob

Prototype
`Std_ReturnType vHsm_Api_ProcessTempJob (uint32 cryIfChannelId, vHsm_Api_TempJobType *tmpJob)`

Parameter	
cryIfChannelId [in]	CryIf channel which shall be used
tmpJob [in]	Pointer to the job which shall be executed
Return code	
Std_ReturnType	Result of the job execution
Functional Description	
Executes a job. The function checks which driver CryIf channel will be used to execute the job.	
Particularities and Limitations	
-	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 5-8 vHsm_Api_ProcessTempJob

5.2.5 vHsm_Api_CancelTempJob

Prototype	
`Std_ReturnType` **`vHsm_Api_CancelTempJob`** `(uint32 cryIfChannelId, vHsm_Api_TempJobType *tmpJob)`	
Parameter	
cryIfChannelId [in]	CryIf channel which shall be used
tmpJob [in]	Pointer to the job which shall be cancelled
Return code	
Std_ReturnType	Result of the job cancellation
Functional Description	
Cancels a job. The function checks which driver CryIf channel will be used to cancel the job.	
Particularities and Limitations	
-	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 5-9 vHsm_Api_CancelTempJob

5.2.6 vHsm_Api_KeyElementGetForCryptoJobs

Prototype	
`Std_ReturnType vHsm_Api_KeyElementGetForCryptoJobs (uint32 cryptoKeyId, uint32 keyElementId, Crypto_30_LibCv_KeyStorageIndexType *resultIndexPtr, uint32 resultLength, uint8 serviceType)`	
Parameter	
cryptoKeyId [in]	Id of the Key which shall be searched for the key element
keyElementId [in]	Id of the element
resultIndexPtr [in]	Pointer to the index variable
resultLength [in]	Length of the element which shall be retrieved
serviceType [in]	SHE service Type.
Return code	
Std_ReturnType	E_OK if request was successful
	CRYPTO_E_KEY_NOT_VALID if the key element was invalid
	CRYPTO_E_KEY_SIZE_MISMATCH if the size of the key element was not as requested
	E_NOT_OK in all other cases
Functional Description	
Get a key element from the key storage for a job. This function gets a key element and returns a return code suitable for job processing functions.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-10 vHsm_Api_KeyElementGetForCryptoJobs

5.2.7 vHsm_Api_KeyElementSet

Prototype	
`Std_ReturnType vHsm_Api_KeyElementSet (uint32 cryptoKeyId, uint32 keyElementId, const uint8 *keyPtr, uint32 keyLength)`	
Parameter	
cryptoKeyId [in]	Id of the Key which shall be set
keyElementId [in]	Id of the key element which shall be set
keyPtr [in]	Data which shall be stored inside the key element
keyLength [in]	Number of bytes which shall be stored inside the key element

Return code	
Std_ReturnType	CRYPTO_E_KEY_WRITE_FAIL if a write once key has already been written,
Std_ReturnType	otherwise the return value of the called KeyElementSet function of the LibCv
Functional Description	
Handles the write once flag before handing over the key operation to the actual LibCv KeyElementSet function.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-11 vHsm_Api_KeyElementSet

5.2.8 vHsm_Api_KeyElementSet_Unchecked

Prototype	
`Std_ReturnType` **`vHsm_Api_KeyElementSet_Unchecked`** `(uint32 cryptoKeyId, uint32 keyElementId, const uint8 *keyPtr, uint32 keyLength)`	
Parameter	
cryptoKeyId [in]	Id of the Key which shall be set
keyElementId [in]	Id of the key element which shall be set
keyPtr [in]	Data which shall be stored inside the key element
keyLength [in]	Number of bytes which shall be stored inside the key element
Return code	
Std_ReturnType	E_OK if request was successful.
	CRYPTO_E_KEY_NOT_AVAILABLE if request failed, the key is not available.
	CRYPTO_E_KEY_SIZE_MISMATCH if request failed, the key element size does not match size of provided data.
Functional Description	
Sets the given key element bytes to the key identified by cryptoKeyId. The key element is not checked for write access flags. This API can be used to set keys internally which would otherwise not be writable from outside.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-12 vHsm_Api_KeyElementSet_Unchecked

5.2.9 vHsm_Api_DisallowKeyUsage

Prototype	
`Std_ReturnType` **`vHsm_Api_DisallowKeyUsage`** `(uint32 cryIfKeyId)`	
Parameter	
cryIfKeyId [in]	Id of the Key which usage shall be disallowed
Return code	
Std_ReturnType	E_OK if request was successful.
	E_NOT_OK if invalid cryIfKeyId was passed.
Functional Description	
Prevents the given key identified by cryIfKeyId from being used. The key is set with access restriction. This API can be used to disallow the usage of keys from outside the vHsm.	
Particularities and Limitations	
-	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 5-13 vHsm_Api_DisallowKeyUsage

5.2.10 vHsm_Api_AllowKeyUsage

Prototype	
`Std_ReturnType` **`vHsm_Api_AllowKeyUsage`** `(uint32 cryIfKeyId)`	
Parameter	
cryIfKeyId [in]	Id of the Key which usage shall be allowed
Return code	
Std_ReturnType	E_OK if request was successful.
	E_NOT_OK if invalid cryIfKeyId was passed.
Functional Description	
Enables the given key identified by cryIfKeyId to be used. The access flag of the key is restored. This API can be used to allow the usage of keys from outside the vHsm.	
Particularities and Limitations	
-	
Call context	
> ANY > This function is Synchronous	

> This function is Reentrant

Table 5-14 vHsm_Api_AllowKeyUsage

5.2.11 vHsm_Init

Prototype	
`void vHsm_Init (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Init function of the vHsm Firmware. This is the first entry point for the vHsm Firmware	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-15 vHsm_Init

5.2.12 vHsm_TriggerSoftwareReset

Prototype	
`void vHsm_TriggerSoftwareReset (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Trigger a software reset.	
Particularities and Limitations	
-	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 5-16 vHsm_TriggerSoftwareReset

5.2.13 vHsm_TriggerSoftwareResetWithIpcClearing

Prototype	
`void` **`vHsm_TriggerSoftwareResetWithIpcClearing`** `(void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Trigger a software reset including clearing of IPC.	
Particularities and Limitations	
-	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 5-17 vHsm_TriggerSoftwareResetWithIpcClearing

5.2.14 vHsm_WriteErrorLog

Prototype	
`void` **`vHsm_WriteErrorLog`** `(uint32 errorId, uint32 errorData)`	
Parameter	
errorId [in]	Id of the error
errorData [in]	Data of the error. VHSM_LOG_DATA_NO_ERROR_DATA indicates not data.
Return code	
void	none
Functional Description	
Write an error entry to the Key Storage. The error entry consists of an id and a data value which will be placed at the beginning of the memory space of the key element. The older entries will be moved to the back. If there is not enough space for old elements, they will be discarded.	
Particularities and Limitations	
-	
Call context	
> TASK|ISR > This function is Synchronous > This function is Reentrant	

Table 5-18 vHsm_WriteErrorLog

5.2.15 vHsm_ExternalIsr

Prototype	
`void vHsm_ExternalIsr (vHsm_CallContextType callContext)`	
Parameter	
callContext [in]	Context of the caller which can be either VHSM_CALL_CONTEXT_ISR1 or VHSM_CALL_CONTEXT_ISR2
Return code	
void	none
Functional Description	
This function is called by the ISR. This function will fetch jobs from the IPC. Depending on the configuration: - the jobs can directly be processed if interrupt processing is enabled for the CryIf channel - a OS task may be activated if context is Cat2 ISR - a switch to ISR Cat2 will be performed to activate an OS Task if context is Cat1 ISR - Only fetching is performed in the ISR	
Particularities and Limitations	
-	
Call context	
> ISR1|ISR2 > This function is Synchronous > This function is not Reentrant	

Table 5-19 vHsm_ExternalIsr

5.2.16 vHsm_InternalIsr

Prototype	
`void vHsm_InternalIsr (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Function which is call by a Cat2 interrupt which has been triggered by vHsm_Hal_TriggerInternalIrq(). OS tasks cannot be triggered from a Cat1 interrupt context. Therefore, a switch is needed to Cat2.	
Particularities and Limitations	
-	
Call context	
> ISR2	

> This function is Synchronous
> This function is not Reentrant

Table 5-20 vHsm_ExternalIsr

5.2.17 vHsm_Api_Ramloop_PrepareExecution

Prototype	
`void vHsm_Api_Ramloop_PrepareExecution (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Prepares vHsm to enter ram loop. Ram loop is entered in next cycle of the main function.	
Particularities and Limitations	
-	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 5-21 vHsm_Api_Ramloop_PrepareExecution

5.2.18 vHsm_Api_Ramloop_Exit

Prototype	
`void vHsm_Api_Ramloop_Exit (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Removes vHsm form ram loop. Ram loop is exited and vHsm runs back in code flash.	
Particularities and Limitations	
-	
Call context	
> ANY > This function is Synchronous	

> This function is Reentrant

Table 5-22 vHsm_Api_Ramloop_Exit

5.3 Services used by vHsm_Core

In the following table services provided by other components, which are used by the vHsm_Core are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
Crypto_30_LibCv	Crypto_30_LibCv_Init Crypto_30_LibCv_InitMemory Crypto_30_LibCv_MainFunction Crypto_30_LibCv_ProcessJob Crypto_30_LibCv_CancelJob Crypto_30_LibCv_<KeyM Functions>
vHsm_Hal	vHsm_Hal_Init vHsm_Hal_InitMemory vHsm_Hal_MainFunction vHsm_Hal_ProcessJob vHsm_Hal_CancelJob vHsm_Hal_<KeyM Functions> vHsm_Hal_SystemInit vHsm_Hal_StartupCallout vHsm_Hal_HandleResetInfo vHsm_Hal_StartApplicationCores vHsm_Hal_TriggerSoftwareReset vHsm_Hal_EnableCodeFlashAccess vHsm_Hal_DisbleCodeFlashAccess vHsm_Hal_MemRand vHsm_Hal_GetCodeFlashAddr vHsm_Hal_Halt_Callout vHsm_Hal_GetIpcAddress vHsm_Hal_GetDynamicMemoryAreaConfig vHsm_Hal_PrepareCryptoJobData vHsm_Hal_PrepareKeyMJobData vHsm_Hal_TriggerInternalIrq vHsm_Hal_EnableInterrupSource vHsm_Hal_DisableInterrupSource
vHsm_Custom	vHsm_Custom_Init vHsm_Custom_InitMemory vHsm_Custom_MainFunction vHsm_Custom_ProcessJob vHsm_Custom_CancelJob vHsm_Custom_<KeyM Functions>
NvM	NvM_MainFunction NvM_Init

Component	API
	NvM_ReadAll NvM_WriteAll NvM_SetRamBlockStatus

Table 5-23 Services used by the vHsm_Core

5.4 Configurable Interfaces

5.4.1 Callout Functions

At its configurable interfaces the vHsm_Core defines callout functions. The declarations of the callout functions are provided by the BSW module, i.e. the vHsm_Core. It is the integrator's task to provide the corresponding function definitions. However, vHsm_Core is providing the vHsm_Callout_Stubs.c during the generation as a starting point. The definitions of the callouts can be adjusted to the system's needs. Some callouts will already be filled according the configuration. As this file can be updated within each new generation, code should only be added in the user blocks of this file as code gets lost otherwise. Alternatively, vHsm_Callout_Stubs.c can be excluded from project, so callouts can be defined in every other file.

The vHsm_Core callout function declarations are described in the following tables:

5.4.1.1 vHsm_StartupCallout

Prototype	
`void vHsm_StartupCallout (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Callout which is called during startup at the very beginning. This function is called from the startup code. Ram does not need to be initialized at this point.	
Particularities and Limitations	
-	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 5-24 vHsm_StartupCallout

5.4.1.2 vHsm_SystemInit_Callout

Prototype
`void vHsm_SystemInit_Callout (void)`

Parameter	
-	-
Return code	
-	-
Functional Description	
Callout which is called directly at the startup of the vHsm. The clock init to the full speed can be performed here to minimize the startup time.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-25 vHsm_SystemInit_Callout

5.4.1.3 vHsm_FirstStartup_Callout

Prototype	
`void vHsm_FirstStartup_Callout (void)`	
Parameter	
-	-
Return code	
-	-
Functional Description	
Callout which indicates the first startup of the vHsm Firmware.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-26 vHsm_FirstStartup_Callout

5.4.1.4 vHsm_InvalidBlockDetected_Callout

Prototype	
`void vHsm_InvalidBlockDetected_Callout (void)`	
Parameter	
-	-

Return code	
-	-
Functional Description	
Callout which is called when a not readable block is detected at startup while performing ReadAll(). This will happen on first startup and should not occur during runtime except when data flash gets corrupted.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-27 vHsm_InvalidBlockDetected_Callout

5.4.1.5 vHsm_InitOne_Callout

Prototype	
`void vHsm_InitOne_Callout (void)`	
Parameter	
-	-
Return code	
-	-
Functional Description	
Callout which is called to initialize memory.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-28 vHsm_InitOne_Callout

5.4.1.6 vHsm_InitTwo_Callout

Prototype	
`void vHsm_InitTwo_Callout (void)`	
Parameter	
-	-

Return code	
-	-
Functional Description	
Callout which is called to initialize modules before NvM read all.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-29 vHsm_InitTwo_Callout

5.4.1.7 vHsm_InitThree_Callout

Prototype	
`void vHsm_InitThree_Callout (void)`	
Parameter	
-	-
Return code	
-	-
Functional Description	
Callout which is called to initialize modules after NvM read all.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-30 vHsm_InitThree_Callout

5.4.1.8 vHsm_NvM_ProcessBlock_Callout

Prototype	
`void vHsm_NvM_ProcessBlock_Callout (void)`	
Parameter	
-	-
Return code	
-	-

Functional Description
Callout which is called while the NvM is processing. This callout can be used for triggering a watchdog.
Particularities and Limitations
-
Call context
> TASK > This function is Synchronous > This function is Reentrant

Table 5-31 vHsm_NvM_ProcessBlock_Callout

5.4.1.9 vHsm_NvMainFunctions_Callout

Prototype	
`void vHsm_NvMainFunctions_Callout (void)`	
Parameter	
-	-
Return code	
-	-
Functional Description	
Callout which is called to trigger the main functions needed to process the NvM.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-32 vHsm_NvMainFunctions_Callout

5.4.1.10 vHsm_MainFunctionCyclic_Callout

Prototype	
`void vHsm_MainFunctionCyclic_Callout (void)`	
Parameter	
-	-
Return code	
-	-

Functional Description
Callout which is called cyclically during the main loop.
Particularities and Limitations
-
Call context
> TASK > This function is Synchronous > This function is Reentrant

Table 5-33 vHsm_MainFunctionCyclic_Callout

5.4.1.11 vHsm_RamLoop_Callout

Prototype	
`Std_ReturnType` **`vHsm_RamLoop_Callout`** `(void)`	
Parameter	
-	-
Return code	
Std_ReturnType	none
Functional Description	
Callout to which is called while the vHsm waits inside the RAM. Can be used to leave RAM loop by returning E_NOT_OK.	
Particularities and Limitations	
The callout must be mapped to the RAM.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-34 vHsm_RamLoop_Callout

5.4.1.12 vHsm_PreRamLoop_Callout

Prototype	
`Std_ReturnType` **`vHsm_PreRamLoop_Callout`** `(void)`	
Parameter	
-	-
Return code	
Std_ReturnType	none

Functional Description
Callout which is called to allow/prevent the vHsm to be sent into RAM.
Particularities and Limitations
-
Call context
> TASK > This function is Synchronous > This function is Reentrant

Table 5-35 vHsm_PreRamLoop_Callout

5.4.1.13 vHsm_FirmwareUpdate_GetMacKey_Callout

Prototype	
`Std_ReturnType` **`vHsm_FirmwareUpdate_GetMacKey_Callout`** `(uint8 *keyPtr, uint32 *length)`	
Parameter	
keyPtr [in]	Pointer to the buffer where 16 bytes of key data shall be stored
length [in, out]	length of the buffer
Return code	
Std_ReturnType	none
Functional Description	
Callout to get the value of the MAC key which is used during the vHsm Firmware Update.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-36 vHsm_FirmwareUpdate_GetMacKey_Callout

5.4.1.14 vHsm_FirmwareUpdate_GetCipherKey_Callout

Prototype	
`Std_ReturnType` **`vHsm_FirmwareUpdate_GetCipherKey_Callout`** `(uint8 *keyPtr, uint32 *length)`	
Parameter	
keyPtr [in]	Pointer to the buffer where 16 bytes of key data shall be stored

length [in, out]	length of the buffer
Return code	
Std_ReturnType	none
Functional Description	
Callout to get the value of the Encryption key which is used during the vHsm Firmware Update.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-37 vHsm_FirmwareUpdate_GetCipherKey_Callout

5.4.1.15 vHsm_Custom_IsSecureBootAllowAutostart_Callout

Prototype
`void vHsm_Custom_IsSecureBootAllowAutostart_Callout (vHsm_SecureBootGroupIterType secureBootGroupIndex, uint32 secureBootGroupId)`
Parameter
secureBootGroupIndex: Index of the secure boot group secureBootGroupId: Id of the secure boot group
Return code
boolean: TRUE if automatic start is allowed, otherwise false to prevent the automatic start of the group and the contained segments.
Functional Description
Callout which is called when a secure boot group is configured to be automatically started and the callout is enabled for this group. This callout can be used to allow or restrict a group from being automatically started.
Particularities and Limitations
-
Call context
> TASK > This function is Synchronous > This function is Reentrant

Table 5-38 vHsm_Custom_IsSecureBootAllowAutostart_Callout

5.4.1.16 vHsm_Custom_IsSecureBootAllowCyclicVerification_Callout

Prototype
`void vHsm_Custom_IsSecureBootAllowCyclicVerification_Callout (vHsm_SecureBootGroupIterType secureBootGroupIndex, uint32 secureBootGroupId)`

Parameter	
secureBootGroupIndex	Index of the secure boot group
secureBootGroupId	Id of the secure boot group
Return code	
boolean	TRUE if automatic start is allowed, otherwise false to prevent the cyclic start of the group and the contained segments.
Functional Description	
Callout which is called when a secure boot group is configured to be cyclic and the callout is enabled for this group. This callout can be used to allow or restrict a group from being cyclically started.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-39 vHsm_Custom_IsSecureBootAllowCyclicVerification_Callout

5.4.1.17 vHsm_Custom_SecureBootSanction_Callout

Prototype	
`Std_ReturnType` **`vHsm_Custom_SecureBootSanction_Callout`** `(uint32 cryptoKeyId, vHsm_SecureBootInfoType* vHsm_SecureBootInfoPtr)`	
Parameter	
cryptoKeyId	Key id of the failed segment
SecureBootInfoPtr	Pointer to the secure boot info containing the information if it is a cyclic verification or if the segment shall be retried if it failed
Return code	
boolean	E_OK if successful, otherwise E_NOT_OK
Functional Description	
This callout is called when a segment has been detected as invalid and the sanction was specified to call this function.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-40 vHsm_Custom_SecureBootSanction_Callout

5.4.1.18 vHsm_SecureBoot_SegmentSuccess_Callout

Prototype	
`Std_ReturnType` **`vHsm_SecureBoot_SegmentSuccess_Callout`** `(uint32 cryptoKeyId, vHsm_SecureBootInfoType* vHsm_SecureBootInfoPtr)`	
Parameter	
cryptoKeyId	Key id of the failed segment
SecureBootInfoPtr	Pointer to the secure boot info containing the information if it is a cyclic verification or if the segment shall be retried if it failed
Return code	
boolean	E_OK if successful, otherwise E_NOT_OK
Functional Description	
This callout is called when a segment has been detected as valid and the sanction.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-41 vHsm_SecureBoot_SegmentSuccess_Callout

5.4.1.19 vHsm_Log_Event_Callout

Prototype	
`Std_ReturnType` **`vHsm_Log_Event_Callout`** `(uint32 logEventId)`	
Parameter	
logEventId	Id of the event
Return code	
boolean	E_OK if successful, otherwise E_NOT_OK
Functional Description	
Callout which is called if an event occurs which has the callout enabled	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-42 vHsm_Log_Event_Callout

5.4.1.20 vHsm_Log_FillLevelThreshold_Callout

Prototype	
`Std_ReturnType` **`vHsm_Log_FillLevelThreshold_Callout`** `(uint32 logEventId)`	
Parameter	
logEventId	Id of the event
Return code	
boolean	E_OK if successful, otherwise E_NOT_OK
Functional Description	
Callout which is called if the configured fill level is reached	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-43 vHsm_Log_FillLevelThreshold_Callout

5.4.1.21 vHsm_Log_GetTimestamp_Callout

Prototype	
`Std_ReturnType` **`vHsm_Log_GetTimestamp_Callout`** `(uint32 logEventId, uint8* timestampPtr, uint32* timestampLengthPtr)`	
Parameter	
logEventId	Id of the event
`timestampPtr`	Pointer to the timestamp which shall to be filled by the callout
`timestampLengthPtr`	Pointer to the length of the timestamp buffer. After calling this function, the number of written bytes needs to be stored there.
Return code	
boolean	E_OK if successful, otherwise E_NOT_OK
Functional Description	
Callout which is called if an event occurs which has the callout enabled	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-44 vHsm_Log_GetTimestamp_Callout

6 Configuration

6.1 Configuration Variants

The vHsm_Core supports the configuration variants

> VARIANT-PRE-COMPILE

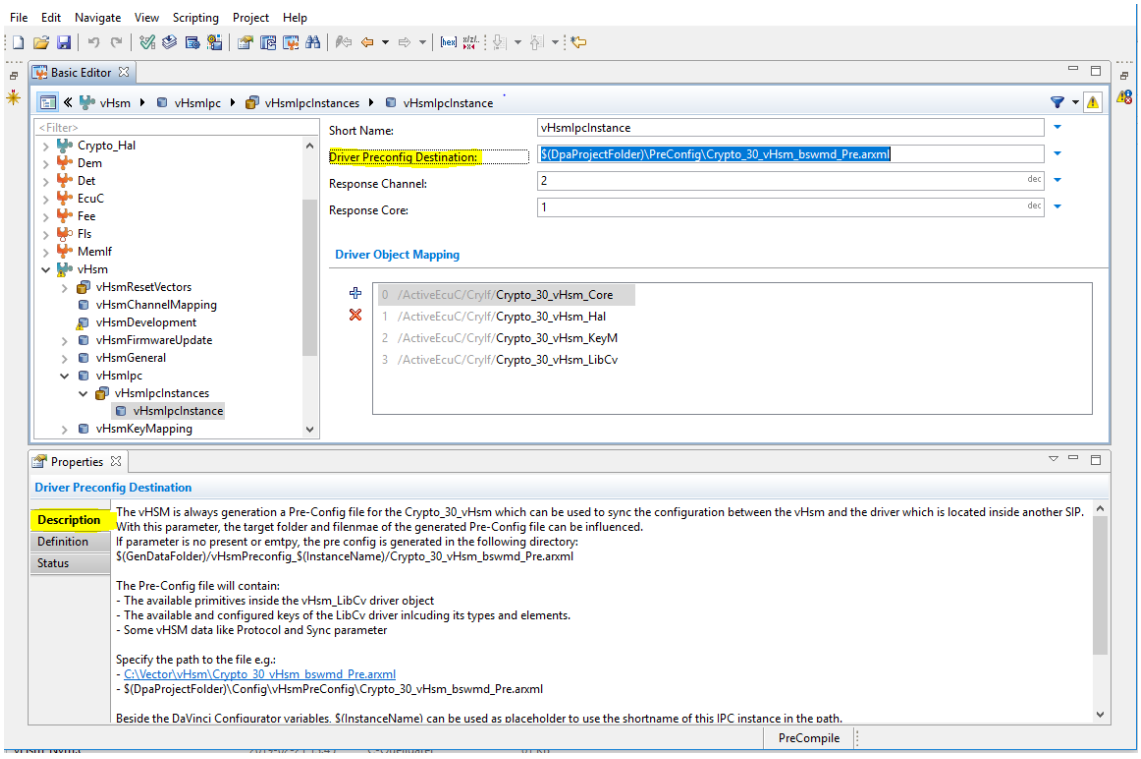
The configuration classes of the vHsm_Core parameters depend on the supported configuration variants. For their definitions please see the [BSW_specific]_bswmd.arxml file.

6.2 Configurable Attributes

In the vHsm_Core the attributes can be configured according to/ with the following methods/ tools:

> Configuration in DaVinci Configurator 5 Pro

The description of each configurable option is described within its online help in the Configurator 5 tool.



Note
A description for each parameter can be found inside DaVinci Cfg5 under the properties tab:

7 Glossary and Abbreviations

7.1 Glossary

Term	Description
IPC	Inter Processor Communication; Shared memory used to exchange data between application core and HSM.
HSM	Hardware Security Module: Dedicated core with own secure memory and hardware accelerators for common cryptographic functions
vHsm	Firmware from Vector Informatik for the HSM Core
HAL	Hardware Abstraction Layer of vHsm Firmware

Table 7-1 Glossary

7.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
DEM	Diagnostic Event Manager
DET	Development Error Tracer
EAD	Embedded Architecture Designer
ECU	Electronic Control Unit
HIS	Hersteller Initiative Software
ISR	Interrupt Service Routine
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
PPORT	Provide Port
RPORT	Require Port
RTE	Runtime Environment
SRS	Software Requirement Specification
SWC	Software Component
SWS	Software Specification

Table 7-2 Abbreviations

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com